第 5 章 openGauss 编译

5.1 实验介绍

openGauss 数据库源代码的编译与调试是开展后续其他实验的工作基础。基于源代码的开发工作实际上就是按照“编辑——编译——测试——调试”这样的步骤循环进行的。“编辑”（edit）是指对源代码就行编辑修改，以达到预定目标；“编译”（compile）是将编辑修改后的源代码整体构建为可执行文件的过程；“测试”（test）包括了安装过程，对安装好的可执行文件进行测试，看是否达到了编辑代码要做到的预定目标；“调试”（debug）是根据测试中发现的问题，查找为什么没有达到预定目标的过程，定位查找到问题原因后，进行编辑修正，开启新一轮的上述循环过程，直到达到要编码实现的预定目标为止。openGauss 作为开源数据库管理系统，其开发过程遵循同样的流程。我们这套实验也是按此步骤，到 openGauss 源代码中去验证数据库系统原理的实现机制。因而，openGauss 的编译和调试是进行后续实验必须掌握的技能。

本章将通过实验介绍如何对 openGauss 源代码进行编译，并对编译后的 openGauss 数据库进行安装；关于 openGauss 的调试将在下一章涉及。

5.2 实验目的

1. 了解 openGauss 数据库的编译环境与配置要求。
2. 掌握 openGauss 源代码的编译和安装过程。
3. 掌握 openGauss 数据库的初始化、启动和连接方法。
4. 通过编译和安装过程深入理解 openGauss 数据库的服务器/客户端运行原理。
5. 掌握 openGauss 的编译和安装过程，为后续实验操作打下基础。

5.3 实验原理

5.3.1 openGauss 编译环境

进行 openGauss 编译需要用到两个组件：openGauss-server 和 binarylibs。openGauss-server 是 openGauss 的主要代码，可以从 openGauss 开源社区获取；binarylibs 是 openGauss 依赖的第三方库文件。由于直接编译 binarylibs 源代码过程复杂费时，推荐从 openGauss 开源社区下载已经编译好的文件。

编译 openGauss 的环境要求包括：

1. 操作系统：CentOS 7.6（x86 架构）或 openEuler-20.03-LTS（aarch64 架构）。
2. 依赖软件：

表 5.1 openGauss 编译环境依赖软件

依赖软件	版本要求

libaio-devel	0.3.109-13
flex	2.5.31 及以上版本
bison	2.7-4
ncurses-devel	5.9-13.20130511
glibc-devel	2.17-111
patch	2.7.1-10
readline-devel	建议版本：7.0-13
lsb_release	4.1

3. 编译器：GCC 编译器。
4. 配置工具：使用 configure 脚本来检测系统的配置和可用性，并生成 makefile 文件。
5. 编译工具：使用 make 命令来编译源代码。

在编译 openGauss 之前，需要确保系统上已安装所有依赖的库和头文件，并且已配置好编译器和其它必要的工具。

5.3.2 Linux 配置与编译工具

在 Linux 系统下，使用 configure 和 make 进行编译是一种常用的方法。

- configure 是一个脚本文件，用于检测系统的配置和可用性，并生成 makefile 文件。它会检测系统上已安装的库、头文件、编译器等信息，并在生成的 makefile 文件中包含这些信息。
- make 是一个命令行工具，用于根据 makefile 文件中的指令编译源代码。它会读取 makefile 文件，并根据文件中的指令编译源代码。 一般来说，makefile 文件会包含编译源代码所需的所有信息，如编译选项、源文件、目标文件等。

在这个流程中，configure 会生成一个根据系统配置的 makefile 文件，然后使用 make 命令来执行编译操作。这种方法可以让程序在不同环境下可以自动适配。

5.4 实验步骤

5.4.1 准备用户和目录

1. 创建用于 openGauss 实验操作的操作系统用户 dblab。

   以 root 用户身份登录主机。该主机可以是任意能给 SSH 远程访问的主机。此处我们使用了名为 eduog 的华为云主机。root 用户登录进云主机后，显示提示符如下：

   ```
   [root@eduog ~]#
   ```

   使用 useradd 命令创建 dblab 用户：

   ```
   [root@eduog ~]# useradd -g dbgrp -u 1001 -d /home/dblab dblab
   ```

   设置 dblab 用户密码：

   ```
   [root@eduog ~]# passwd dblab
   ```

   以 dblab 用户身份登录：

   ```
   [dblab@eduog ~]$ su - dblab
   ```

   创建用于编译 openGauss 的目录 opengauss-compile：

   ```
   [dblab@eduog ~]$ mkdir opengauss-compile
   ```

进入到 opengauss-compile 目录：

```
[dblab@eduog ~]$ cd opengauss-compile/
[dblab@eduog opengauss-compile]$
```

5.4.2 下载相关文件

1. 在本套实验中，使用 openGauss 5.0.0 版本。访问下面链接地址，下载 openGauss-server v5.0.0 的源代码（2023 年 3 月 31 日发布），如图 5.1 所示：
https://gitee.com/opengauss/openGauss-server/tags
https://gitee.com/opengauss/openGauss-server/repository/archive/v5.0.0

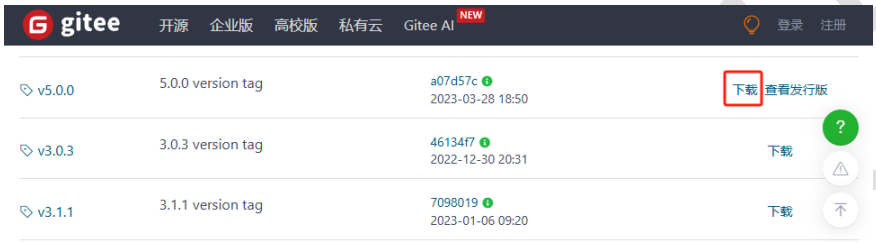

图 5.1 下载 openGauss-server 源代码

将下载的 zip 压缩包文件 openGauss-server-v5.0.0.zip 使用文件传输工具（如 SFTP 客户端）上传到云主机。

2. 下载 openGauss 开源社区提供的已编译好的第三方库二进制文件包（为 tar.gz 格式的压缩包），如

图 5.2 所示。注意，需要选择所用操作系统对应的文件包（如我们使用的是 openEuler ARM 版本操作系统，要下载 openEuler_arm 版本文件包）。

图 5.2 下载 openGauss 第三方库二进制文件包 binarylibs

可以在云主机上使用 wget 命令进行下载：

```
[dblab@eduog opengauss-compile]$ wget https://opengauss.obs.cn-south-1.myhuaweicloud.com/5.0.0/binarylibs/openGauss-third_party_binarylibs_openEuler_arm.tar.gz
```

https://opengauss.obs.cn-south-1.myhuaweicloud.com/5.0.0/binarylibs/openGauss-third_party_binarylibs_openEuler_arm.tar.gz

3. 解压缩 openGauss 源代码包和第三方库包：

   解压 openGauss 源代码包 openGauss-server-v5.0.0.zip：

```
[dblab@eduog opengauss-compile]$ unzip openGauss-server-v5.0.0.zip
```

   重命名 openGauss 源代码包目录为 opengauss-server：

```
[dblab@eduog opengauss-compile]$ mv openGauss-server-v5.0.0 opengauss-server
```

   解压第三方库二进制包 openGauss-third_party_binarylibs.tar.gz：

```
[dblab@eduog opengauss-compile]$ tar -zxvf openGauss-third_party_binarylibs_openEuler_arm.tar.gz
```

   重命名第三方库二进制包目录为 binarylibs：

```
[dblab@eduog opengauss-compile]$ mv openGauss-third_party_binarylibs_openEuler_arm binarylibs
```

   在 root 用户下使用命令 `yum install -y tree` 下载 tree 包，接着使用 tree 命令查看 opengauss-compile 目录的树形结构：

```
[dblab@eduog opengauss-compile]$ tree -L 1
.
├── binarylibs
├── opengauss-server
├── openGauss-server-v5.0.0.zip
└── openGauss-third_party_binarylibs_openEuler_arm.tar.gz
```

5.4.3 配置环境变量

1. 使用 vim 编辑器编辑~/.bashrc 文件，添加环境变量：

```
[dblab@eduog ~]$ vim ~/.bashrc
```

在.bashrc 文件中配置如下环境变量：

```
export CODE_BASE=/home/dblab/opengauss-compile/opengauss-server # Path of the
openGauss-server file
export BINARYLIBS=$CODE_BASE/../binarylibs
export GAUSSHOME=$CODE_BASE/dest
export GCC_PATH=$BINARYLIBS/buildtools/ gcc7.3
export CC=$GCC_PATH/gcc/bin/gcc
export CXX=$GCC_PATH/gcc/bin/g++
export
LD_LIBRARY_PATH=$GAUSSHOME/lib:$GCC_PATH/gcc/lib64:$GCC_PATH/isl/lib:$GCC_PATH/mpc/li
b/:$GCC_PATH/mpfr/lib/:$GCC_PATH/gmp/lib/:$LD_LIBRARY_PATH
export PATH=$GAUSSHOME/bin:$GCC_PATH/gcc/bin:$PATH
```

配置的各环境变量说明如下：

- CODE_BASE：openGauss 源代码所在路径
- BINARYLIBS：已编译好的第三方库文件所在路径
- GAUSSHOME：编译后进行 openGauss 安装的路径
- GCC_PATH：gcc 编译器工具的路径
- CC：gcc 编译器命令的路径
- CXX：g++编译器命令的路径
- LD_LIBRARY_PATH：用来指定系统在查找动态链接库时应该搜索的目录
- PATH：用来指定系统在查找可执行文件时应该搜索的目录

2. 执行 source 命令，使环境变量生效

```
[dblab@eduog ~]$ source ~/.bashrc
```

5.4.4 配置 swap 分区

由于在 openGauss 编译过程中内存消耗较大，建议添加 swap 分区，以增加虚拟内存。如能成功执行步骤 2.4.5 则此步骤可略过，若不能成功执行则需返回此步骤配置 swap 分区。

1. 以 root 用户登录云主机，执行以下命令，创建 4G 大小的 swap 文件：

使用 dd 命令，创建文件/var/swapfile，读写块大小为 1M 字节，块个数为 4096，因而总大小为 4G 字节。

```
[root@eduog ~]# dd if=/dev/zero  of=/var/swapfile  bs=1MiB  count=4096
```

使用 mkswap 命令，将/var/swapfile 转变为 swap 文件：

```
[root@eduog ~]# mkswap  /var/swapfile
```

关于 dd 命令和 mkswap 命令的详细用法，可参见 Linux 命令文档。

2. 编辑文件/etc/fstab，添加 swap 分区配置，使得每次开机时 swap 配置均能够生效

```
[root@eduog ~]# vim /etc/fstab
```

在/etc/fstab 文件中添加如下一行：

```
/var/swapfile   swap    swap    sw  0   0
```

3. 使用 swapon 命令启用/etc/fstab 配置的 swap 分区

```
[root@eduog ~]# swapon -a
```

5.4.5 进行编译

1. 现在可以进行 openGauss 编译。进入 opengauss-server 目录，准备进行编译：

```
[dblab@eduog ~]$ cd opengauss-compile/opengauss-server
```

2. 执行 configure 配置脚本，进行编译前的配置。为了进行调试，配置为 debug 版本：

```
[dblab@eduog opengauss-server]$ ./configure --gcc-version=7.3.0 CC=g++ CFLAGS='-O0' -
-prefix=$GAUSSHOME --3rd=$BINARYLIBS --enable-debug --enable-cassert --enable-thread-
safety --with-readline --without-zlib
```

这里执行的 configure 脚步的各个参数解释如下：

- --gcc-version=7.3.0：使用 GCC 版本 7.3.0 编译。
- CC=g++：指定编译器为 g++。
- CFLAGS='-O0'：设置编译器优化级别为 0。
- --prefix=$GAUSSHOME：安装程序的路径为 $GAUSSHOME。
- --3rd=$BINARYLIBS：使用预编译的第三方库文件，路径为 $BINARYLIBS。
- --enable-debug：启用调试模式。
- --enable-cassert：启用 C 语言的 assert 语句。
- --enable-thread-safety：启用线程安全性。
- --with-readline：使用 readline 库。
- --without-zlib：不使用 zlib 库。

如果配置脚本执行过程中出现报错，可能原因包括：缺少依赖库或工具，编译器版本问题、环境变量配置问题等。需要根据报错提示进行检查修复。

3. 执行 make 命令，进行编译。make 命令的-j4 选项指定并行编译并采用 4 个进程。

```
[dblab@eduog opengauss-server]$ make -j4
```

编译需要执行一段时间，控制台会滚动输出编译命令消息，请耐心等待编译过程结束，编译过程截图如图 5.3 所示。图 5.3 左侧显示的是 make 命令执行中的编译命令输出；右侧显示的是采用 top 命令查看系统资源的使用情况。

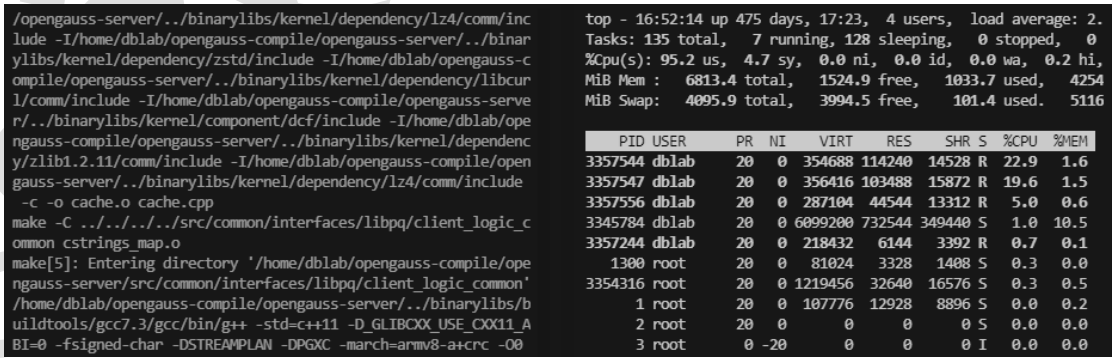

图 5.3 使用 make 命令进行 openGauss 源代码编译

在 make 编译执行过程中，可能报出各种错误。以下是一些常见的报错及其解决方法：

- "No rule to make target"：请检查 Makefile 文件中是否存在相应的编译规则。

- "No such file or directory"：请检查源文件是否存在，并确保路径是正确的。
- "undefined reference"：请检查是否缺少链接库或链接库的路径是否正确。
- "permission denied"：请确保当前用户有写入文件的权限。
- "error X"：请检查源代码是否有语法错误，并根据错误信息进行修改。

如果遇到错误，可以尝试查看错误信息，在网上搜索解决方案。

4. 如编译过程中没有遇到任何报错，则编译成功，最后一行输出“All of openGauss successfully made. Ready to install.”，如

```
g++ -std=c++11 -D_GLIBCXX_USE_CXX11_ABI=0 -fsigned-char -DSTREAMPLAN -DPGXC -march=armv8-a+crc -O0 -Wall -Wpointer-arith -Wno-write
-strings -fnon-call-exceptions -fno-common -freg-struct-return -pipe -Wendif-labels -Wmissing-format-attribute -Wformat-security -f
no-strict-aliasing -fwrapv -g -DENABLE_GSTRACE -fno-aggressive-loop-optimizations -Wno-attributes -fno-omit-frame-pointer -fno-expe
nsive-optimizations -Wno-unused-but-set-variable -fstack-protector -Wl,-z,relro,-z,now -Wl,-z,noexecstack -std=c++14 -pthread  -D_R
EENTRANT -D_THREAD_SAFE -D_POSIX_PTHREAD_SEMANTICS -fpic -shared -o regress.so regress.o -L../../../src/common/port -pthread -L/hom
e/dblab/opengauss-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/zlib1.2.11/comm/lib -I/home/dblab/opengaus
s-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/zlib1.2.11/comm/include -L/home/dblab/opengauss-compile/op
enGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/zstd/lib -I/home/dblab/opengauss-compile/openGauss-server-v5.0.0/../bi
narylibs-v5.0.0/kernel/dependency/zstd/include -L/home/dblab/opengauss-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/
platform/Huawei_Secure_C/comm/lib -L/home/dblab/opengauss-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/op
enssl/comm/lib -L/home/dblab/opengauss-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/buildtools/libstd/gcc7.3.0/comm/lib -L/
home/dblab/opengauss-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/libcgroup/comm/lib -L -L/home/dblab/ope
ngauss-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/unixodbc/lib -L/home/dblab/opengauss-compile/openGaus
s-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/libobs/comm/lib -L/home/dblab/opengauss-compile/openGauss-server-v5.0.0/../b
inarylibs-v5.0.0/kernel/dependency/kerberos/comm/lib -L../../../src/gstrace//common -L/home/dblab/opengauss-compile/openGauss-serve
r-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/numactl/comm/lib -L/home/dblab/opengauss-compile/openGauss-server-v5.0.0/../binaryl
ibs-v5.0.0/kernel/dependency/libcurl/comm/lib -L/usr1/gauss_jenkins/jenkins/workspace/openGauss/thirdparty/dependency/libxml2/insta
ll_comm/lib -L/home/dblab/opengauss-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/libxml2/comm/lib  -L/hom
e/dblab/opengauss-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/jemalloc/debug/lib
chmod +x ./smartmatch.pl
make[2]: Leaving directory '/home/dblab/opengauss-compile/openGauss-server-v5.0.0/src/test/regress'
make[1]: Leaving directory '/home/dblab/opengauss-compile/openGauss-server-v5.0.0/src'
All of openGauss successfully made. Ready to install.
```

```
g++ -std=c++11 -D_GLIBCXX_USE_CXX11_ABI=0 -fsigned-char -DSTREAMPLAN -DPGXC -march=armv8-a+crc -O0 -Wall -Wpointer-arith -Wno-write
-strings -fnon-call-exceptions -fno-common -freg-struct-return -pipe -Wendif-labels -Wmissing-format-attribute -Wformat-security -f
no-strict-aliasing -fwrapv -g -DENABLE_GSTRACE -fno-aggressive-loop-optimizations -Wno-attributes -fno-omit-frame-pointer -fno-expe
nsive-optimizations -Wno-unused-but-set-variable -fstack-protector -Wl,-z,relro,-z,now -Wl,-z,noexecstack -std=c++14 -pthread  -D_R
EENTRANT -D_THREAD_SAFE -D_POSIX_PTHREAD_SEMANTICS -fpic -shared -o regress.so regress.o -L../../../src/common/port -pthread -L/hom
e/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel/dependency/zlib1.2.11/comm/lib -I/home/dblab/opengauss-compile/open
gauss-server/../binarylibs/kernel/dependency/zlib1.2.11/comm/include -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs
/kernel/dependency/zstd/lib -I/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel/dependency/zstd/include -L/home/d
blab/opengauss-compile/opengauss-server/../binarylibs/kernel/platform/Huawei_Secure_C/comm/lib -L/home/dblab/opengauss-compile/open
gauss-server/../binarylibs/kernel/dependency/openssl/comm/lib -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/buildt
ools/libstd/gcc7.3.0/comm/lib -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel/dependency/libcgroup/comm/lib -
L -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel/dependency/unixodbc/lib -L/home/dblab/opengauss-compile/ope
ngauss-server/../binarylibs/kernel/dependency/libobs/comm/lib -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel
/dependency/kerberos/comm/lib -L../../../src/gstrace//common -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel/
dependency/numactl/comm/lib -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel/dependency/libcurl/comm/lib -L/us
r1/gauss_jenkins/jenkins/workspace/openGauss/thirdparty/dependency/libxml2/install_comm/lib -L/home/dblab/opengauss-compile/opengau
ss-server/../binarylibs/kernel/dependency/libxml2/comm/lib  -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel/d
ependency/jemalloc/debug/lib
chmod +x ./smartmatch.pl
make[2]: Leaving directory '/home/dblab/opengauss-compile/opengauss-server/src/test/regress'
make[1]: Leaving directory '/home/dblab/opengauss-compile/opengauss-server/src'
All of openGauss successfully made. Ready to install.
[dblab@euog opengauss-server]$
```

5. 图 5.4 左图所示。

```
g++ -std=c++11 -D_GLIBCXX_USE_CXX11_ABI=0 -fsigned-char -DSTREAMPLAN -DPGXC -march=armv8-a+crc -O0 -Wall -Wpointer-arith -Wno-write
-strings -fnon-call-exceptions -fno-common -freg-struct-return -pipe -Wendif-labels -Wmissing-format-attribute -Wformat-security -f
no-strict-aliasing -fwrapv -g -DENABLE_GSTRACE -fno-aggressive-loop-optimizations -Wno-attributes -fno-omit-frame-pointer -fno-expe
nsive-optimizations -Wno-unused-but-set-variable -fstack-protector -Wl,-z,relro,-z,now -Wl,-z,noexecstack -std=c++14 -pthread  -D_R
EENTRANT -D_THREAD_SAFE -D_POSIX_PTHREAD_SEMANTICS -fpic -shared -o regress.so regress.o -L../../../src/common/port -pthread -L/hom
e/dblab/opengauss-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/zlib1.2.11/comm/lib -I/home/dblab/opengaus
s-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/zlib1.2.11/comm/include -L/home/dblab/opengauss-compile/op
enGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/zstd/lib -I/home/dblab/opengauss-compile/openGauss-server-v5.0.0/../bi
narylibs-v5.0.0/kernel/dependency/zstd/include -L/home/dblab/opengauss-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/
platform/Huawei_Secure_C/comm/lib -L/home/dblab/opengauss-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/op
enssl/comm/lib -L/home/dblab/opengauss-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/buildtools/libstd/gcc7.3.0/comm/lib -L/
home/dblab/opengauss-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/libcgroup/comm/lib -L -L/home/dblab/ope
ngauss-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/unixodbc/lib -L/home/dblab/opengauss-compile/openGaus
s-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/libobs/comm/lib -L/home/dblab/opengauss-compile/openGauss-server-v5.0.0/../b
inarylibs-v5.0.0/kernel/dependency/kerberos/comm/lib -L../../../src/gstrace//common -L/home/dblab/opengauss-compile/openGauss-serve
r-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/numactl/comm/lib -L/home/dblab/opengauss-compile/openGauss-server-v5.0.0/../binaryl
ibs-v5.0.0/kernel/dependency/libcurl/comm/lib -L/usr1/gauss_jenkins/jenkins/workspace/openGauss/thirdparty/dependency/libxml2/insta
ll_comm/lib -L/home/dblab/opengauss-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/libxml2/comm/lib  -L/hom
e/dblab/opengauss-compile/openGauss-server-v5.0.0/../binarylibs-v5.0.0/kernel/dependency/jemalloc/debug/lib
chmod +x ./smartmatch.pl
make[2]: Leaving directory '/home/dblab/opengauss-compile/openGauss-server-v5.0.0/src/test/regress'
make[1]: Leaving directory '/home/dblab/opengauss-compile/openGauss-server-v5.0.0/src'
All of openGauss successfully made. Ready to install.
```

```
g++ -std=c++11 -D_GLIBCXX_USE_CXX11_ABI=0 -fsigned-char -DSTREAMPLAN -DPGXC -march=armv8-a+crc -O0 -Wall -Wpointer-arith -Wno-write
-strings -fnon-call-exceptions -fno-common -freg-struct-return -pipe -Wendif-labels -Wmissing-format-attribute -Wformat-security -f
no-strict-aliasing -fwrapv -g -DENABLE_GSTRACE -fno-aggressive-loop-optimizations -Wno-attributes -fno-omit-frame-pointer -fno-expe
nsive-optimizations -Wno-unused-but-set-variable -fstack-protector -Wl,-z,relro,-z,now -Wl,-z,noexecstack -std=c++14 -pthread  -D_R
EENTRANT -D_THREAD_SAFE -D_POSIX_PTHREAD_SEMANTICS -fpic -shared -o regress.so regress.o -L../../../src/common/port -pthread -L/hom
e/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel/dependency/zlib1.2.11/comm/lib -I/home/dblab/opengauss-compile/open
gauss-server/../binarylibs/kernel/dependency/zlib1.2.11/comm/include -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs
/kernel/dependency/zstd/lib -I/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel/dependency/zstd/include -L/home/d
blab/opengauss-compile/opengauss-server/../binarylibs/kernel/platform/Huawei_Secure_C/comm/lib -L/home/dblab/opengauss-compile/open
gauss-server/../binarylibs/kernel/dependency/openssl/comm/lib -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/buildt
ools/libstd/gcc7.3.0/comm/lib -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel/dependency/libcgroup/comm/lib -
L -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel/dependency/unixodbc/lib -L/home/dblab/opengauss-compile/ope
ngauss-server/../binarylibs/kernel/dependency/libobs/comm/lib -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel
/dependency/kerberos/comm/lib -L../../../src/gstrace//common -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel/
dependency/numactl/comm/lib -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel/dependency/libcurl/comm/lib -L/us
r1/gauss_jenkins/jenkins/workspace/openGauss/thirdparty/dependency/libxml2/install_comm/lib -L/home/dblab/opengauss-compile/opengau
ss-server/../binarylibs/kernel/dependency/libxml2/comm/lib  -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel/d
ependency/jemalloc/debug/lib
chmod +x ./smartmatch.pl
make[2]: Leaving directory '/home/dblab/opengauss-compile/opengauss-server/src/test/regress'
make[1]: Leaving directory '/home/dblab/opengauss-compile/opengauss-server/src'
All of openGauss successfully made. Ready to install.
[dblab@eduog opengauss-server]$
```

图 5.4 openGauss 源代码编译成功

6. 【可选】清理编译过程中生成的文件，使目录恢复到编译前的状态，准备进行下一次重新编译。如编译成功或无需清理文件，则不要执行此命令。

```
[dblab@eduog opengauss-server]$ make clean
```

5.4.6 进行安装

1. 将编译好的 openGauss 二进制文件进行安装，执行如下命令：

```
[dblab@eduog opengauss-server]$ make install
```

如 openGauss 安装成功，最后一行显示“openGauss installation complete.”，如

```
_jenkins/jenkins/workspace/openGauss/thirdparty/dependency/libxml2/install_comm/lib -L/home/dblab/opengauss-compile/opengauss-serve
r/../binarylibs/kernel/dependency/libxml2/comm/lib  -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel/dependenc
y/jemalloc/debug/lib  -L../../src/common/interfaces/libpq -lpq -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kerne
l/dependency/unixodbc/lib -lodbc
/bin/mkdir -p '/home/dblab/opengauss-compile/opengauss-server/dest/lib/postgresql'
/bin/mkdir -p '/home/dblab/opengauss-compile/opengauss-server/dest/share/postgresql/extension'
/bin/mkdir -p '/home/dblab/opengauss-compile/opengauss-server/dest/share/postgresql/extension'
/bin/sh ../../config/install-sh -c -m 755  dblink.so '/home/dblab/opengauss-compile/opengauss-server/dest/lib/postgresql/dblink.so'
/bin/sh ../../config/install-sh -c -m 644 ./dblink.control '/home/dblab/opengauss-compile/opengauss-server/dest/share/postgresql/ex
tension/'
/bin/sh ../../config/install-sh -c -m 644 ./dblink--1.0.sql ./dblink--unpackaged--1.0.sql  '/home/dblab/opengauss-compile/opengauss
-server/dest/share/postgresql/extension/'
make[1]: Leaving directory '/home/dblab/opengauss-compile/opengauss-server/contrib/dblink'
openGauss installation complete.
[dblab@eduog opengauss-server]$
```

图 5.5 所示。


```
_jenkins/jenkins/workspace/openGauss/thirdparty/dependency/libxml2/install_comm/lib -L/home/dblab/opengauss-compile/opengauss-serve
r/../binarylibs/kernel/dependency/libxml2/comm/lib  -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kernel/dependenc
y/jemalloc/debug/lib  -L../../src/common/interfaces/libpq -lpq -L/home/dblab/opengauss-compile/opengauss-server/../binarylibs/kerne
l/dependency/unixodbc/lib -lodbc
/bin/mkdir -p '/home/dblab/opengauss-compile/opengauss-server/dest/lib/postgresql'
/bin/mkdir -p '/home/dblab/opengauss-compile/opengauss-server/dest/share/postgresql/extension'
/bin/mkdir -p '/home/dblab/opengauss-compile/opengauss-server/dest/share/postgresql/extension'
/bin/sh ../../config/install-sh -c -m 755  dblink.so '/home/dblab/opengauss-compile/opengauss-server/dest/lib/postgresql/dblink.so'
/bin/sh ../../config/install-sh -c -m 644 ./dblink.control '/home/dblab/opengauss-compile/opengauss-server/dest/share/postgresql/ex
tension/'
/bin/sh ../../config/install-sh -c -m 644 ./dblink--1.0.sql ./dblink--unpackaged--1.0.sql  '/home/dblab/opengauss-compile/opengauss
-server/dest/share/postgresql/extension/'
make[1]: Leaving directory '/home/dblab/opengauss-compile/opengauss-server/contrib/dblink'
openGauss installation complete.
[dblab@eduog opengauss-server]$
```


图 5.5 openGauss 安装成功

5.4.7 初始化数据库

安装好的 openGauss 数据库需要进行初始化之后才能运行数据库服务器。初始化

openGauss 数据库需要使用 gs_initdb 命令。该命令创建了数据库所需的目录结构和配置文件，并初始化了系统目录和配置文件，具体包括：创建数据库目录、生成系统表、创建默认数据库和模板数据库。初始化数据库时会生成大量的系统表和视图。初始化数据库过程中生成的数据库包括：

- template0 数据库：是 openGauss 提供的最初始的备份数据库，当需要时可用 template0 作为模板生成“干净”的数据库。
- template1 数据库：是一个模板数据库，当以后再创建一个新的数据库时，template1 数据库里的所有内容都会拷贝到新数据库中。
- postgres 数据库：是一个内置的、专门用于管理系统元数据的数据库。它包含了系统中所有数据库、表、视图、规则、存储过程等元数据的信息。这些信息在 openGauss 中被用来维护数据库的一致性和完整性，并由系统内置的管理工具来维护和使用。这个数据库对于管理员来说是不可见的，并且不能被用户直接访问。

gs_initdb 命令执行格式如下：

```
gs_initdb [options] [pgdata]
```

其中，[options] 是可选参数，[pgdata] 是数据库存储目录。

gs_initdb 命令常用选项包括：

- --pgdata 或-D：指定数据库存储目录。
- --nodename：初始化的节点名称。
- --encoding 或-E：为新数据库设置编码格式。

执行如下 gs_initdb 命令，初始化 openGauss 数据库：

```
[dblab@eduog opengauss-server]$ gs_initdb --pgdata=$GAUSSHOME/data --nodename=dblab --encoding=utf-8
```

其中，"--pgdata=$GAUSSHOME/data"参数指定了数据库实例的存储目录，"$GAUSSHOME"是 5.4.3 节中配置的环境变量。"--nodename=dblab"参数指定了数据库实例的名称。"--encoding=utf-8"参数指定了数据库实例的编码方式。运行这条命令会在"$GAUSSHOME/data"目录下创建一个名为"dblab"的数据库实例，该实例的编码方式为 UTF-8。

openGauss 数据库初始化成功后，将显示输出“Success. You can now start the database server of single node using: …”，提示初始化数据库成功，可以使用 gaussdb 或 gs_ctl 命令启动数据库服务器，如下所示。

```
loading hstore extension ... ok
loading security plugin ... ok
update system tables ... ok
creating snapshots catalog ... ok
vacuuming database template1 ... ok
copying template1 to template0 ... ok
copying template1 to postgres ... ok
freezing database template0 ... ok
freezing database template1 ... ok
freezing database postgres ... ok

WARNING: enabling "trust" authentication for local connections. You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the next time you run gs_initdb.
```

```
Success. You can now start the database server of single node using:
    gaussdb -D / home/dblab/opengauss-compile/opengauss-server/dest/data --single_node
or
    gs_ctl start -D /home/dblab/opengauss-compile/opengauss-server/dest/data -z single_node
-l logfile
```

5.4.8 启动数据库服务器

确保在启动 openGauss 数据库服务器之前已经通过 gs_initdb 成功初始化了数据库实例。初始化数据库成功后，可使用 gs_ctl 命令启动数据库服务器。gs_ctl 是 openGauss 提供的数据库服务控制工具，可以用来启动、停止、重启数据库服务以及查询数据库服务状态。

常用的 gs_ctl 命令有：

- gs_ctl start：启动数据库服务器
- gs_ctl stop：停止数据库服务器
- gs_ctl restart：重启数据库服务器
- gs_ctl status：查看数据库服务器状态
- gs_ctl reload：重新加载配置文件

这些命令可以通过参数来指定数据库实例目录、日志文件、集群模式等信息。通过这些命令可以方便地管理 openGauss 数据库服务器。

1. 使用 gs_ctl 命令启动 openGauss 数据库服务器的基本语法是：

   ```
   gs_ctl -D [pgdata-directory] start
   ```

   其中，"-D <pgdata-directory>" 参数指定了数据库实例所在目录。

   执行如下命令，启动 openGauss 数据库服务器：

   ```
   [dblab@eduog opengauss-server]$ gs_ctl start -D $GAUSSHOME/data -Z single_node -l
   logfile
   ```

   这个命令会启动 openGauss 单节点数据库服务器，其数据库实例目录位于 $GAUSSHOME/data，并且在 logfile 文件中记录运行日志。

   gs_ctl 命令行参数说明：

   - -D $GAUSSHOME/data：指定了数据库实例所在目录。
   - -Z single_node：指定了启动单节点模式。
   - -l logfile：指定了日志文件的文件名，日志内容将记录在该文件中。

   启动 openGauss 数据库服务器成功后，输出如下所示：

   ```
   [2025-02-23 15:19:41.272][3345781][][gs_ctl]: gs_ctl started,datadir is
   /home/dblab/opengauss-compile/opengauss-server/dest/data
   [2025-02-23 15:19:41.294][3345781][][gs_ctl]: waiting for server to start...
   .
   [2025-02-23 15:19:42.308][3345781][][gs_ctl]:  done
   [2025-02-23 15:19:42.308][3345781][][gs_ctl]: server started (/home/dblab/opengauss-
   compile/opengauss-server/dest/data)
   ```

2. 可使用 gs_ctl status 命令查看数据库服务器的运行状态：

   ```
   [dblab@eduog opengauss-server]$ gs_ctl status -D $GAUSSHOME/data


   [2025-02-23 15:21:40.301][3345894][][gs_ctl]: gs_ctl status,datadir is
   ```

```
/home/dblab/opengauss-compile/opengauss-server/dest/data
    gs_ctl: server is running (PID: 3345784)
/home/dblab/opengauss-compile/opengauss-server/dest/bin/gaussdb "-D"
"/home/dblab/opengauss-compile/opengauss-server/dest/data"
```

3. 【可选】如需停止数据库服务器，可以使用如下命令：

```
[dblab@eduog opengauss-server]$ gs_ctl stop -D $GAUSSHOME/data -Z single_node -l
logfile
```

5.4.9 连接数据库

使用命令行客户端 gsql 连接 openGauss 数据库服务器。gsql 是 openGauss 数据库的客户端工具，它是一个命令行工具，支持 SQL 语言，与 openGauss 数据库进行交互。它可以用于执行 SQL 查询、管理数据库对象、管理数据库实例等。gsql 还提供了一些脚本命令，用于完成一些特定的任务，例如创建数据库、创建表、导入数据等。

1. 执行如下命令，使用 gsql 客户端连接数据库服务器：

```
[dblab@eduog opengauss-server]$ gsql postgres -r
```

其中，postgres 是要连接的数据库名称，-r 选项表示打开 gsql 客户端的行编辑功能，运行使用退格键等进行命令编辑。

如连接成功，会进入 gsql 交互式命令行界面，如下所示。

```
[dblab@eduog opengauss-server]$ gsql postgres -r
gsql ((openGauss 5.0.0 build ) compiled at 2025-02-23 14:49:07 commit 0 last mr debug)
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.
openGauss=#
```

按照要求，在 gsql 中执行任何命令之前，要首先修改数据库用户 dblab 的密码，执行以下 SQL 语句：

```
openGauss=# ALTER ROLE "dblab" PASSWORD '替换为自行设定的密码';


openGauss=# ALTER ROLE "dblab" PASSWORD 'dblab@123';
ALTER ROLE
```

2. 在 gsql 中，测试执行 SQL 语句是否可以执行成功。执行以下 SQL 语句，返回 openGauss 版本号，可以看到返回内容中包括编译 openGauss 的时间戳信息，如下所示。

```
openGauss=# SELECT VERSION();
                                    version
-----------------------------------------------------------------------------------
(openGauss 5.0.0 build ) compiled at 2025-02-23 14:49:07 commit 0 last mr debug on
aarch64-unknown-linux-gnu, compiled by g++ (GCC)7.3.0, 64-bit
(1 row)
```

3. openGauss 默认设置 session_timeout = 10 min，也就是当连接进入数据库后，若十分钟内没有任何操作，数据库就会执行超时处理。对于实际生产应用来说，这一点是合理的设计，但是对于学习者来说超出超时时间后就需要重连数据库，使得操作变得繁琐。可以进入数据库系统中使用 set session_timeout=0;这种方法仅对当下对

话产生作用，使用 show session_timeout; 可以查看当前超时时间的设置。

aUSS数据库实验教

5.5 实验结果

【请按照要求完成实验操作并粘贴截图】

1. 按照步骤成功完成 openGauss 源代码编译，并安装 openGauss 数据库。

2. 完成 openGauss 数据库初始化，并启动数据库服务器。

3. 完成 gsql 客户端连接数据库服务器并成功执行“SELECT version();”语句。

auss数据库实验教

5.6 实验讨论与总结

【请将实验中遇到的问题描述、解决办法与思考讨论列在下面，并对本实验进行总结。】

aus数据库实验教

第 6 章 openGauss 调试

6.1 实验介绍

本实验将实践通过 VS Code 进行 openGauss 源代码调试的步骤。首先，介绍如何进行 VS Code 开发环境配置，对 openGauss 进行远程开发；然后，配置在 VS Code 中 openGauss 的调试环境，实践通过在源代码内设置断点进行单步调试；最后，实践以 attach 方式进行 openGauss 的调试过程。本实验通过添加代码，实现在执行 CREATE TABLE 语句时，输出一个循环变量值，以掌握 VS Code 开发环境以及 openGauss 单步调试过程。本实验践行了“编辑——编译——测试——调试”的迭代开发过程。读者可以在后续实验中不断通过实践体会该过程，为系统软件开发积累经验。

6.2 实验目的

1. 掌握 VS Code 远程开发环境的配置操作，能够满足 openGauss 源代码开发要求。
2. 掌握 VS Code 中 openGauss 调试环境的配置。
3. 掌握 VS Code 中 openGauss 源代码单步调试操作过程。
4. 掌握 VS Code 中以 attach 方式进行 openGauss 源代码调试操作过程。
5. 理解 openGauss 的“编辑——编译——测试——调试”迭代开发过程。

6.3 实验原理

6.3.1 VS Code 远程开发

在本实验教程中均使用 VS Code 作为源代码开发环境。VS Code 是 Visual Studio Code 的简称，是由 Microsoft 开发的免费、开源、跨平台的代码编辑器。它支持多种编程语言，并拥有丰富的功能，如：

- IntelliSense: 代码完成、代码片段和建议。
- 调试: 步骤调试代码以查找错误。
- 代码导航: 定义跳转、查找所有引用等。
- 集成终端: 在不切换窗口的情况下运行命令行工具。
- Git 集成: 跟踪代码变更并与他人协作。
- 可扩展性: 安装扩展以获得额外功能。
- 可定制的界面和主题。

使用 VS Code 进行远程开发是通过 VS Code 的"Remote-SSH"扩展包实现的。该扩展包允许在本地的 VS Code 编辑器中连接到远程服务器，并在远程服务器上进行开发。Remote-SSH 扩展包通过在本地计算机和远程服务器之间建立 SSH 连接，将远程服务器的文件和功能集成到 VS Code 编辑器中。这使得远程开发变得更加便捷，使开发者可以在本地使用熟悉的编辑环境进行远程服务器上的开发工作。

openGauss数据库实验

使用 VS Code 和 Remote-SSH 扩展包进行远程开发的基本步骤包括:

1. 安装 Remote-SSH 扩展包:可以在 VS Code 的扩展市场中搜索并安装 Remote-SSH 扩展包。
2. 配置 SSH:在本地计算机上,需要配置 SSH 以连接到远程服务器。这通常涉及生成 SSH 密钥对,并将公钥添加到远程服务器的 SSH 授权文件中,以通过 SSH 协议安全地连接到远程服务器。
3. 添加远程主机:在 VS Code 中,可以打开命令面板(Ctrl + Shift + P),搜索并选择"Remote-SSH: Add New SSH Host..."。在弹出的输入框中,提供远程服务器的连接信息,包括远程主机的 IP 地址和 SSH 密钥的位置。
4. 连接到远程服务器:添加了远程主机之后,可以在 VS Code 的左下角找到一个绿色的远程连接按钮。单击此按钮,它将显示添加的远程主机列表。选择要连接的远程主机,VS Code 将使用 SSH 协议与远程服务器建立连接。
5. 远程开发:远程连接建立后,可以在 VS Code 中像本地开发一样使用远程服务器。可以打开远程服务器上的文件夹或工作区,并在本地 VS Code 编辑器中进行编辑、保存和调试操作。所有更改都将在远程服务器上生效,并且可以使用本地的编辑器功能来运行命令、插件和调试工具。

6.3.2 VS Code 远程调试

使用 VS Code 进行 openGauss 远程调试时,可以选择 launch 和 attach 两种调试方式。

1. Launch 调试方式
   - 工作原理:使用 launch 方式进行调试时,在本地计算机上启动的调试会话将通过 SSH 连接连接到远程服务器,并在远程服务器上启动调试目标进程。对于 openGauss,该进程为 gaussdb。
   - 过程:首先,需要在 VS Code 中配置调试器,并指定远程服务器上的调试目标。然后,可以通过设置断点、监视变量等方式在本地 VS Code 编辑器中设置调试配置。一旦启动调试会话,VS Code 将在远程服务器上运行程序,并将调试信息传输回本地 VS Code,这样能够观察和控制调试过程。
2. Attach 调试方式
   - 工作原理:使用 attach 方式进行调试时,可以连接到远程服务器上已经在运行的程序,而无需重新启动程序。即可以在远程服务器上启动程序,然后通过 VS Code 连接到正在运行的进程,以进行调试。
   - 过程:与 launch 方式不同,attach 方式需要在远程服务器上启动程序,然后通过 VS Code 的调试功能将调试器连接到该进程。在本地 VS Code 编辑器中,可以设置断点和监视变量,并开始观察和控制远程进程的调试过程。

6.4 实验步骤

6.4.1 配置 VS Code 远程开发环境

通过下面步骤,对云主机 eduog 上的 openGauss 源代码,配置 VS Code 远程开发环境:

1. 在 VS Code 中安装“Remote-SSH”扩展包。打开 VS Code 左侧工具栏的“Extensions”

openGauss数据库实验

界面，搜索并安装“Remote-SSH”扩展包，如图 6.1 所示。

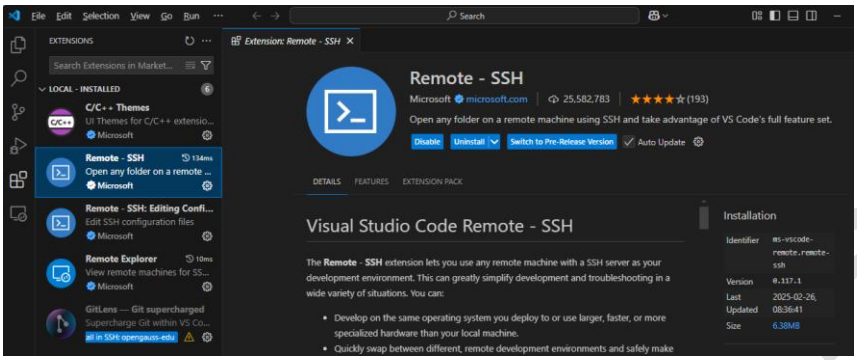

图 6.1 搜索 Remote-SSH 插件并安装

2. 打开 VS Code 左侧工具栏的“Remote Explorer”界面。在“Remote Explorer”（）中选择“Remotes (Tunnels/SSH)”，选择配置“Configure”按钮，如图 6.2 所示。

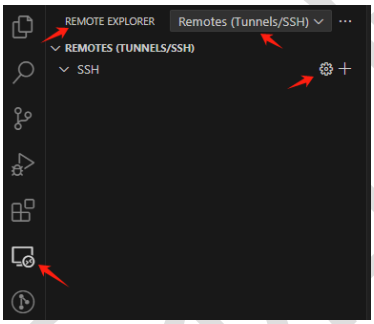

图 6.2 选择 SSH Targets 并配置

3. 在弹出的窗口中，选择<用户目录>\.ssh\config，打开 SSH 配置文件进行编辑，如图 6.3 所示。也可以直接使用快捷键“Ctrl + Shift + P”打开命令面板，选择“Remote-SSH: Open Configuration File”，打开 SSH 配置文件进行编辑。

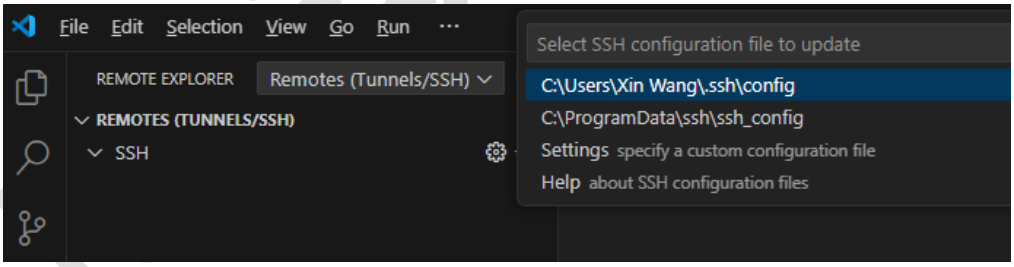

图 6.3 选择并打开 SSH 配置文件

4. 编辑 SSH 配置文件。其中，IdentityFile 参数的值应替换为读者自己私钥文件在本机的保存路径。如图 6.4 所示。

```
Host opengauss-edu
    HostName 124.70.[redacted]
    User dblab
    Port 22
    IdentityFile "C:\[redacted]ssh-login\xinwang-private-key"
```

图 6.4 编辑 SSH 配置文件

SSH 配置文件的参数含义说明如下：

```
Host <远程主机名称，名称自定>
    HostName <远程主机 IP>
    User <登录用户名>
    Port 22 <SSH 端口号，默认 22>
    IdentityFile <SSH 私钥在本机的路径>
```

5. 在使用 SSH 连接远程主机之前，应确保本机具有 ssh 命令。如在 Windows 系统下，在命令行窗口，输入 ssh，显示如图 6.5 所示。如本机不具有 ssh 命令，则可按照网上文档的步骤说明，在本机操作系统上安装 ssh 命令。

```
C:\Users\[redacted]>ssh
usage: ssh [-46AaCfGgKkMNnqsTtVvXxYy] [-B bind_interface]
           [-b bind_address] [-c cipher_spec] [-D [bind_address:]port]
           [-E log_file] [-e escape_char] [-F configfile] [-I pkcs11]
           [-i identity_file] [-J [user@]host[:port]] [-L address]
           [-l login_name] [-m mac_spec] [-O ctl_cmd] [-o option] [-p port]
           [-Q query_option] [-R address] [-S ctl_path] [-W host:port]
           [-w local_tun[:remote_tun]] destination [command]
```

图 6.5 查询本机是否具有 SSH

6. 在进行 ssh 连接之前，请将本机公钥添加进远程主机 dblab 用户下的.ssh/authorized_keys 文件中。
7. 测试本机 ssh 命令可否连接远程主机。命令格式如下：

```
ssh dblab@远程主机 IP -i "私钥文件的本机路径"
```

如出现如图 6.6 中的提示，说明需要限制本机私钥文件的权限，具体操作可参考：https://www.jianshu.com/p/a1314a9448fa

```
C:\Users\Xin Wang>ssh dblab@[redacted] -i "C:\xinwang\ssh-login\xinwang-private-key"

Authorized users only. All activities may be monitored and reported.
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@         WARNING: UNPROTECTED PRIVATE KEY FILE!          @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Permissions for 'C:\\xinwang\\ssh-login\\xinwang-private-key' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
Load key "C:\\xinwang\\ssh-login\\xinwang-private-key": bad permissions
dblab@[redacted]'s password:
```

图 6.6 测试连接远程主机

8. 确保 SSH 命令连接远程主机登录成功，如图 6.7 所示：

```
[dblab@eduog ~]$ client_loop: send disconnect: Connection reset by peer

C:\Users\Xin Wang>ssh dblab@124.70.[redacted] -i "C:\[redacted]ssh-login\xinwang-private-key"

Authorized users only. All activities may be monitored and reported.

        Welcome to Huawei Cloud Service

Last login: Tue Jan [redacted] 2023 from 61.136.[redacted]


Welcome to 4.19.90-2003.4.0.0036.oe1.aarch64

System information as of time:  Tue Jan [redacted] CST 2023

System load:    0.02
Processes:      130
Memory used:    18.4%
Swap used:      76.5%
Usage On:       33%
IP address:     192.168.0.44
Users online:   2



[dblab@eduog ~]$
```

图 6.7 SSH 命令连接远程主机成功

9. 在“Remote Explorer”面板的“REMOTES (TUNNELS/SSH)”菜单的“SSH”子菜单下面，右键点击 opengass-edu 远程主机选项，在弹出的右键菜单中点击“Connect in Current Window”，如图 6.8 所示。

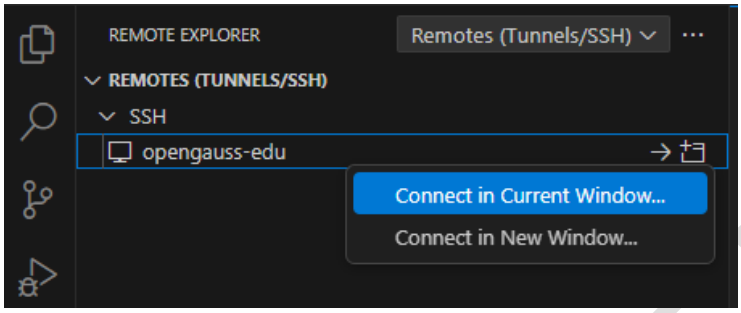

图 6.8 在当前窗口中打开到远程主机的 SSH 连接

10. 在弹出的提示菜单中，选择远程主机的操作系统“Linux”选项。如图 6.9 所示。

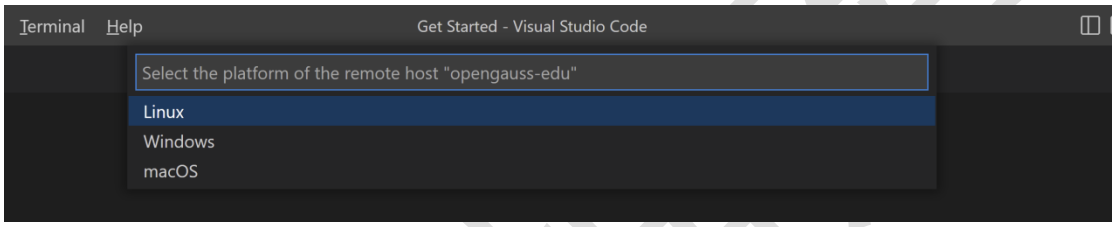

图 6.9 选择远程主机的操作系统

11. VS Code 连接远程主机成功后，Remote Explorer 面板中的 opengass-edu 远程主机选项图标会变为绿色并显示 connected 字样，如图 6.10 所示。

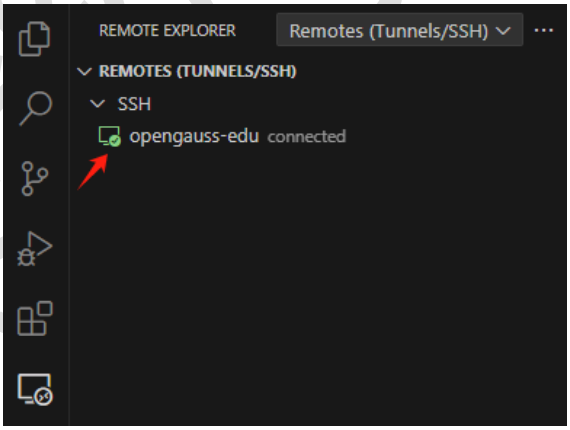

图 6.10 窗口左下角显示 SSH 连接成功

12. 接下来要打开远程主机上 openGauss 源代码文件夹。选择“File”菜单的“Open Folder...”菜单项，打开 opengauss-server 文件夹，如图 6.11 所示。

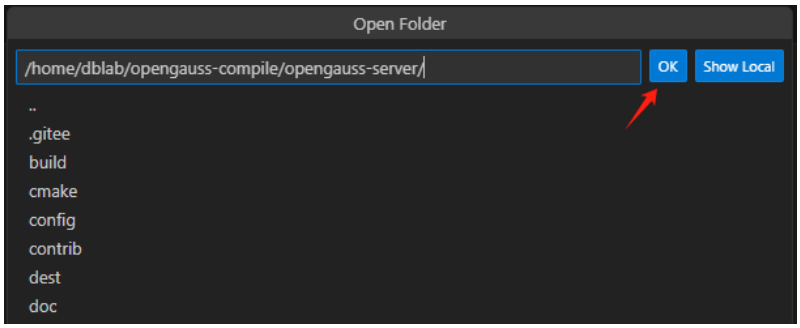

图 6.11 打开远程主机上的 openGauss 源代码文件夹

13. 可以在 EXPLORER 面板中打开某个源代码文件，如 src/common/backend/catalog/catalog.cpp，VS Code 如图 6.12 所示。

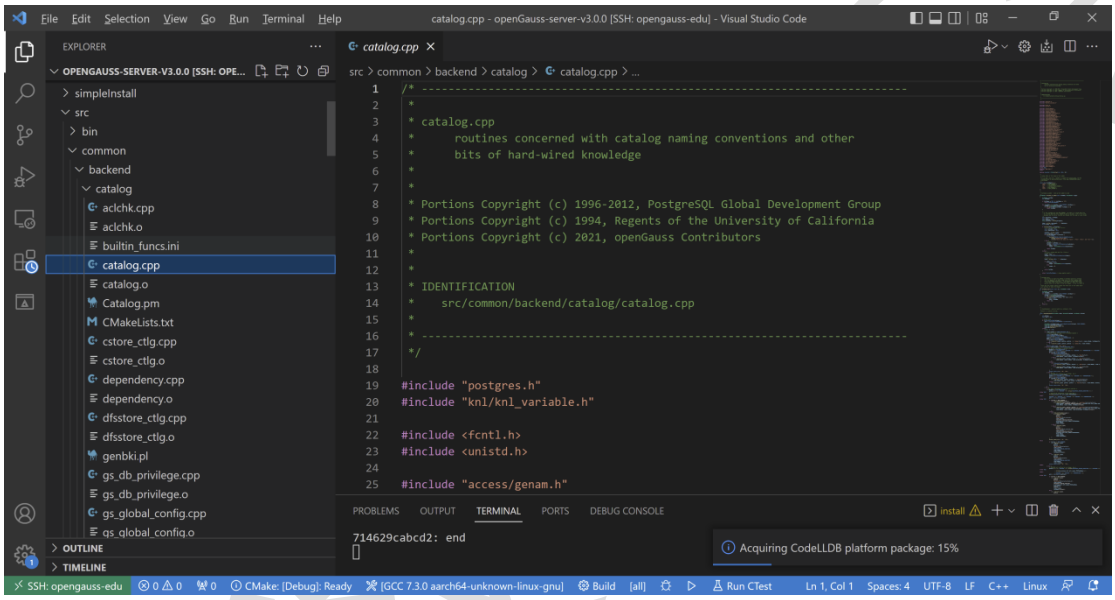

图 6.12 openGauss 项目源码展示

14. 转到左侧工具栏的“Extensions”界面，在“EXTENSIONS: MARKETPLACE”面板中搜索扩展包，选择“Install in SSH: <远程主机名>”，在远程主机上安装 VS Code 的 C/C++ Extension Pack 扩展包。如图 6.13 所示。注意，与前面安装“Remote-SSH”扩展包不同，前者是在本机上安装扩展包，这里是在远程主机上安装扩展包。

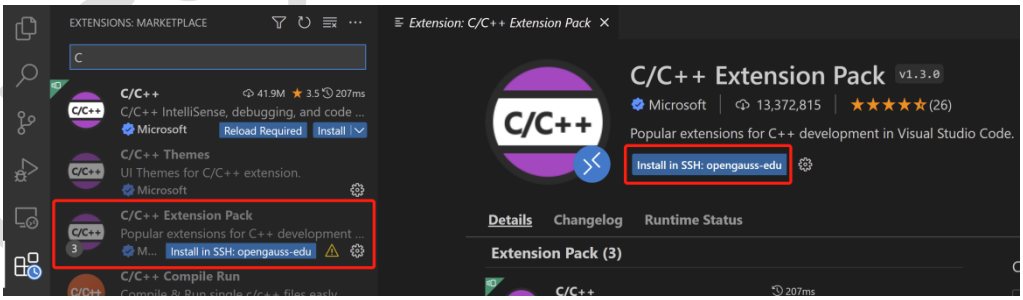

图 6.13 在远程主机上安装 C/C++ Extension Pack 扩展包

C/C++ Extension Pack 扩展包是一组用于增强 VS Code 中 C/C++编程体验的扩展。它包括 Microsoft 的 C/C++扩展、C++ IntelliSense、调试工具和代码格式选项等扩展。其提供的具体功能包括：

- C++ IntelliSense：C++语法和库函数的自动补全、签名帮助和快速信息。
- 调试：使用 VS Code 调试器调试 C/C++代码，支持多线程应用和核心转储。

- 代码导航：C/C++代码的定义、引用查找和符号查看。
- 代码格式化：根据指定的编码风格自动格式化 C/C++代码。
- 代码片段：使用代码片段快速插入常见的 C/C++代码模式。
- 重构：在 C/C++代码上执行重构，例如重命名变量。
- Linting：Lint C/C++代码，检测潜在错误并提出建议。
- 代码评论：使用单个键盘快捷键轻松评论和取消评论 C/C++代码。
- 构建和任务集成：直接从 VS Code 编辑器运行构建和任务，例如编译代码。

6.4.2 openGauss 调试环境配置

1. 确保远程主机上已安装了 gdb 调试器。可以执行命令“gdb -v”输出 gdb 版本信息进行测试，如果系统中已安装了 gdb，则下所示。

```
[dblab@eduog ~]$ gdb -v
GNU gdb (GDB) Euleros 8.3.1-11.oel
Copyright (C) 2019 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later http://gnu.org/licenses/gpl.html
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

2. 在远程主机上进行 GDB 配置。确保 VS Code 已经通过 SSH 成功连接上远程主机，在 VS Code 中，在 openGauss 源代码目录下，新建.vscode 文件夹，在该文件夹下新建 launch.json 文件，文件内容如下：

```
{
    "configurations": [
        {
            "name": "gaussdb",
            "type": "cppdbg",
            "request": "launch",
            "program": "${workspaceFolder}/dest/bin/gaussdb",
            "args": [ "-D", "$GAUSSHOME/data" ],
            "cwd": "${workspaceFolder}",
            "setupCommands": [
                {
                    "text": "handle SIGUSR2 nostop noprint"
                },
            ],
        },
    ]
}
```

该 JSON 文件是用于配置 VS Code 的 C++调试器（cppdbg）的调试会话的启动配置文件。其中的各个参数具体解释如下：

- "name": "gaussdb"：这是配置的名称，即 "gaussdb"。
- "type": "cppdbg"：这是配置的类型，即使用 C++ 调试器（cppdbg）。

- "request": "launch"：这表示请求启动调试会话。
- "program": "${workspaceFolder}/dest/bin/gaussdb"：这是要调试的程序的路径，即 "${workspaceFolder}/dest/bin/gaussdb"。
- "args": ["-D", "$GAUSSHOME/data"]：这是传递给程序的参数，即 "-D $GAUSSHOME/data"。
- "cwd": "${workspaceFolder}"：这是程序的工作目录，即 "${workspaceFolder}"。
- "setupCommands": [{ "text": "handle SIGUSR2 nostop noprint" }]：这是在启动调试会话前要执行的命令，即处理 SIGUSR2 信号，不停止并不输出信息。

6.4.3 openGauss 单步调试

下面在 openGauss 的一个源代码文件中添加几行代码来测试上一小节配置的 VS Code 调试环境。这里我们选择在 heap.cpp 文件的 heap_create_with_catalog 函数末尾添加一个简单的 for 循环语句，在 gsql 客户端输出循环变量。同时演示在 VS Code 中设置断点进行单步调试的过程。

1. 在 VS Code 中，打开文件 src/common/backend/catalog/heap.cpp。在函数 heap_create_with_catalog 末尾，“return relid;”语句之前，增加如下代码：

```
/* [ DBLAB ========================================== */
for (int i = 0; i < 3; i++) {
    ereport(INFO, (0, errmsg("i = %d", i)));
}
/* DBLAB ] ========================================== */
```

添加代码后，如图 6.14 所示。

```
    if (pg_partition_desc) {
        heap_close(pg_partition_desc, RowExclusiveLock);
    }

    /* [ DBLAB ========================================== */
    for (int i = 0; i < 3; i++) {
        ereport(INFO, (0, errmsg("i = %d", i)));
    }
    /* DBLAB ] ========================================== */

    return relid;
}
```

图 6.14 添加代码并设置断点

因 openGauss 代码量巨大，为了将实验添加的代码与 openGauss 源代码相区分，也为了便于查找实验代码，我们约定实验添加代码均放在下面两行注释之间。

```
/* [ DBLAB ========================================== */
// 添加的实验代码
/* DBLAB ] ========================================== */
```

此处添加的代码非常简单，只是使用一个 for 循环，输出循环变量的值。因为要在 gsql 客户端上输出消息，使用的是 openGauss 提供的错误消息处理相关函数 ereport 和 errmsg。

- ereport 函数：用于报告错误，它的作用是生成一条错误消息。

- errmsg 函数：返回错误消息的字符串。

ereport 和 errmsg 函数是 openGauss 中的两个重要函数，用于处理和记录错误消息，帮助开发人员调试程序。

2. 在添加代码的 for 循环上增加一个断点，单击 3138 行号之前的空白处，出现一个红色圆点，如图 6.14 所示。
3. 编译添加的代码并安装：

```
[dblab@eduog opengauss-server]$ make -j4
…
[dblab@eduog opengauss-server]$ make install
…
```

4. 确保没有其他 openGauss 数据库服务器在执行。执行命令，停止之前启动的数据库服务器。

```
[dblab@eduog opengauss-server]$ gs_ctl stop -D $GAUSSHOME/data -Z single_node -l logfile
```

5. 在 VS Code 中，执行“Run”菜单的“Start Debugging”命令，以 gdb 调试模式启动 openGauss 数据库服务器进程 gaussdb，如图 6.15 所示。可以看到，以调试模式启动 gaussdb 完成后，VS Code 下方的状态栏变为橙黄色

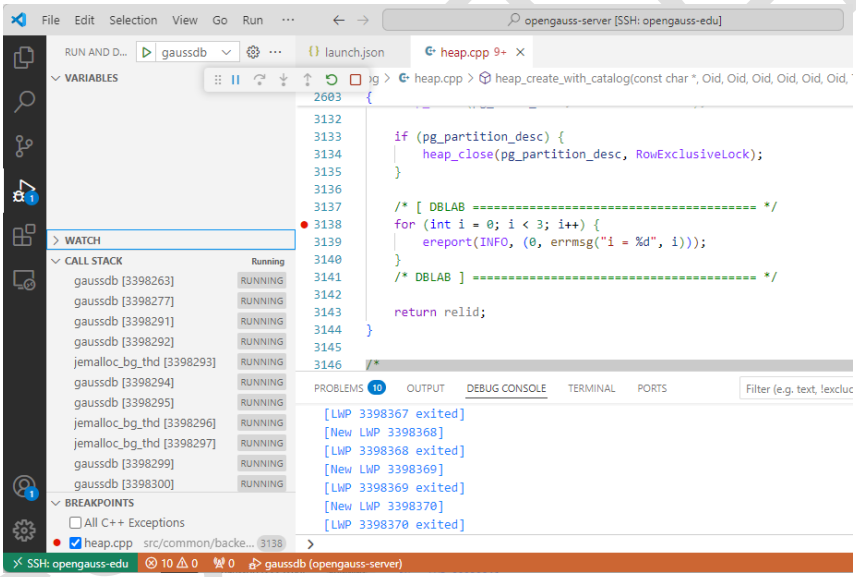

图 6.15 VS Code 以调试模式启动 openGauss 服务器进程 gaussdb

6. 转到 SSH 客户端，执行 gsql 命令，连接以调试模式启动的 gaussdb 进程：

```
[dblab@eduog opengauss-server]$ gsql postgres -r
gsql ((openGauss 5.0.0 build ) compiled at 2025-02-23 16:50:22 commit 0 last mr  debug)
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

openGauss=#
```

7. 在 gsql 中，输入 SQL 语句并执行，测试 heap.cpp 中设置的断点是否生效：

```
openGauss=# CREATE TABLE t ( a int );
```

该语句创建名为 t 的表，该表只有一个 int 类型的列 a。

8. 观察发现，VS Code 调试界面发生变化，调试模式的 gaussdb 进程停在了 heap.cpp 的断点位置，3138 行成为了执行暂停的当前行，“VARIABLES”窗口中的变量、“CALL

STACK”窗口中的函数调用栈、“BREAKPOINTS”窗口中的断点都有了相应的内容。同时观察，gsql 客户端也停住了，并没有返回执行 SQL 语句的结果，证明断点设置生效。如图 6.16 所示。

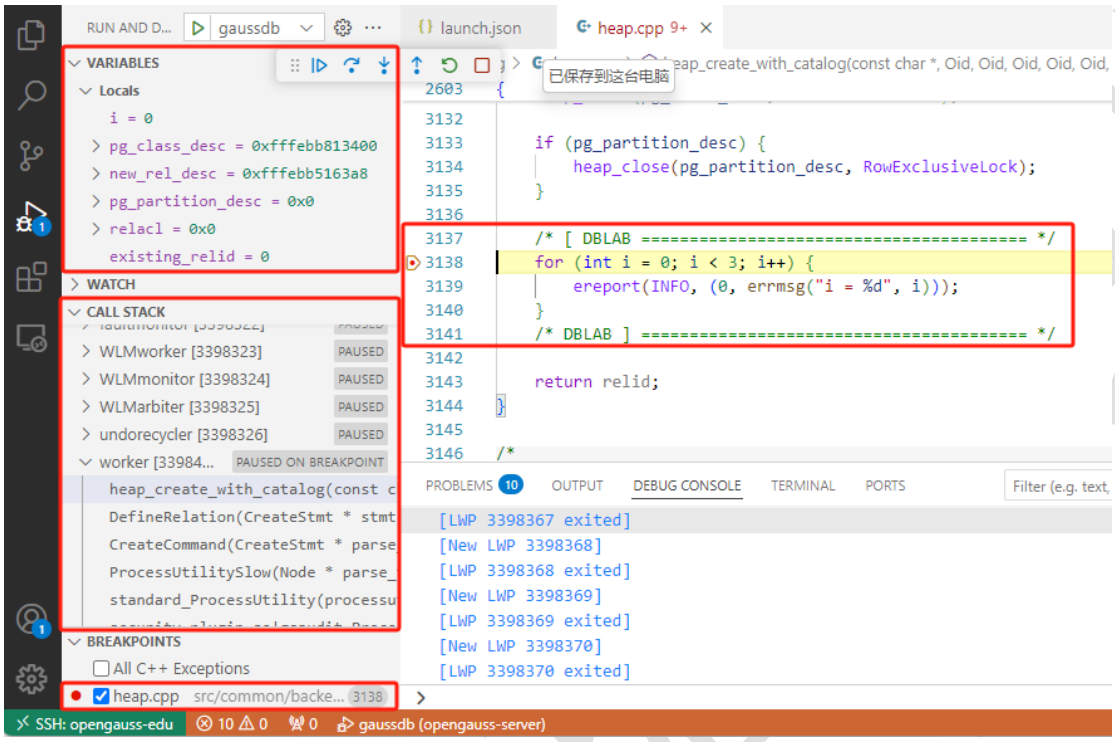

图 6.16 VS Code 调试 openGauss 设置断点生效

9. 使用调试工具栏进行单步执行调试添加的代码。单击“Step Over”按钮，执行当前行。如图 6.17 所示。

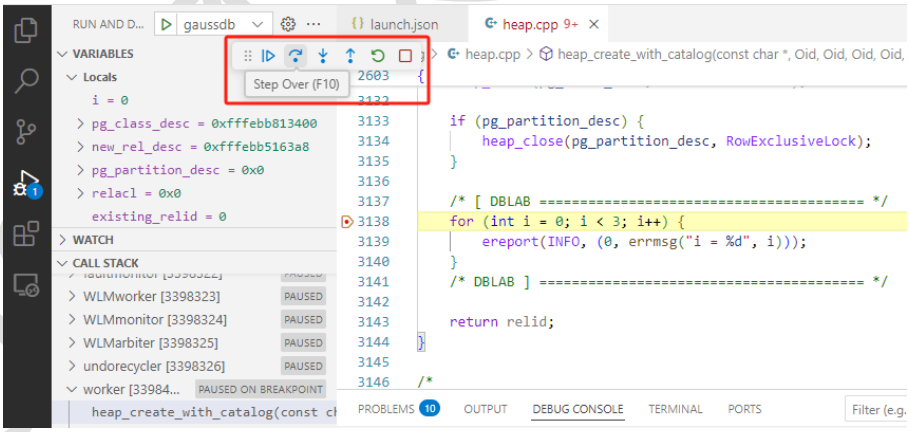

图 6.17 在 VS Code 中进行单步执行调试代码

10. 继续单击“Step Over”按钮，进行单步执行。观察变量 i 的变化以及 gsql 中输出的变化。如图 6.18 所示。

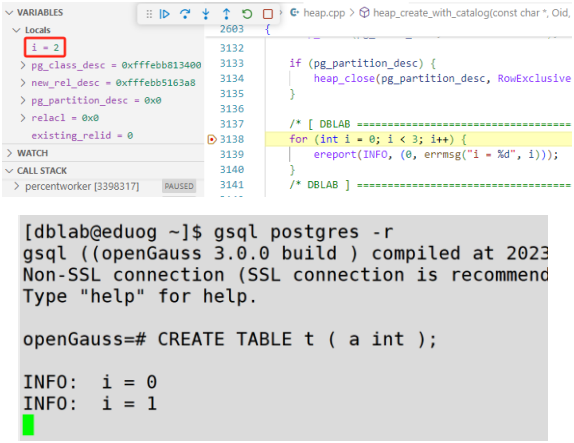

图 6.18 单步执行调试代码时变量的变化以及 gsql 的输出

11. 单击调试工具栏上的“Continue”按钮，使 gaussdb 进行继续执行后续操作。此时，gsql 客户端执行的 SQL 语句会返回结果，如下所示：

```
openGauss=# CREATE TABLE t ( a int );


INFO:  i = 0
INFO:  i = 1
INFO:  i = 2
CREATE TABLE
openGauss=#
```

12. 删除刚创建的表 t 并退出 gsql 客户端：

```
openGauss=# DROP TABLE t;
DROP TABLE
openGauss=# \q
[dblab@eduog opengauss-server]$
```

13. 单击调试工具栏上的“Stop”按钮，停止 VS Code 中 gaussdb 进程的调试。
14. 去掉设置的断点。在 heap.cpp 中断点所在的 3138 行处，单击断点红色圆点，该红色圆点消失。
15. 注释掉添加的代码。选中添加的代码（不包括收尾的注释行），使用快捷键“Ctrl + /”，为选中的整段代码添加注释（再次使用“Ctrl + /”会去掉整段注释，可以使用该快捷键进行添加和删除整段注释的切换），如图 6.19 所示。

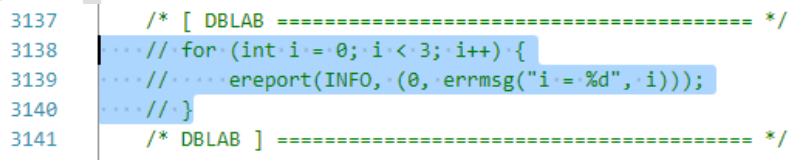

图 6.19 在 VS Code 为整段代码添加注释

16. 停止 openGauss 数据库服务器。

6.4.4 以 attach 方式进行 openGauss 调试

VS Code 支持 launch 和 attach 两种调试模式。Lauch 模式是由 VS Code 启动一个独立

的具有 debug 模式的程序；而 attach 模式是 VS Code 会附加于一个已经启动的程序上，即该程序本身不是通过 VS Code 启动的。以上的 openGauss 调试步骤均是以 launch 方式进行的，本节以 attach 方式进行 openGauss 调试。

1. 在远程主机上进行 GDB 配置。在 VS Code 中，在 openGauss 源代码目录下，打开.vscode 目录下的 launch.json 文件。在 6.4.2 节第 2 步已添加配置的基础上，在“configurations”中继续添加一项以 attach 方式调试的配置，添加后文件内容如下：

```
{
    "configurations": [
        {
            "name": "gaussdb",
            "type": "cppdbg",
            "request": "launch",
            "program": "${workspaceFolder}/dest/bin/gaussdb",
            "args": [ "-D", "$GAUSSHOME/data" ],
            "cwd": "${workspaceFolder}",
            "setupCommands": [
                {
                    "text": "handle SIGUSR2 nostop noprint"
                },
            ],
        },
        {
            "name": "gaussdb (attach)",
            "type": "cppdbg",
            "request": "attach",
            "program": "${workspaceFolder}/dest/bin/gaussdb",
            "processId": "${command:pickProcess}",
            "MIMode": "gdb",
            "miDebuggerPath": "/usr/bin/gdb",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                },
                {
                    "description": "ignore SIGUSR2 signal",
                    "text": "handle SIGUSR2 nostop noprint pass"
                },
                {
                    "description":  "Set Disassembly Flavor to Intel",
                    "text": "-gdb-set disassembly-flavor intel",
                    "ignoreFailures": true
                }
```

```
            ],
        },
    ]
}
```

注意到，与之前的 launch 调试方式不同，新添加的配置里面 request 参数值为 attach，表示该配置项是用于支持 attach 调试方式的。

2. 执行 6.4.3 节中的步骤 1、2、3。
3. 启动 openGauss 数据库服务器。执行如下命令：

```
[dblab@eduog opengauss-server]$ gs_ctl start -D $GAUSSHOME/data -Z single_node -l logfile
```

4. 在 VS Code 中，转到左侧工具栏的“Run and Debug”界面，选择 attach 调试方式，如图 6.20 所示。选择 ▷（即“Start Debugging”）。在“Attach to process”列表中查找并选择“gaussdb”进程，如图 6.21 所示。这时，VS Code 调试功能会附加到已经运行到 openGauss 数据库服务器进程 gaussdb 上，之后即可开始以 attach 方式进行调试，如图 6.22 所示。

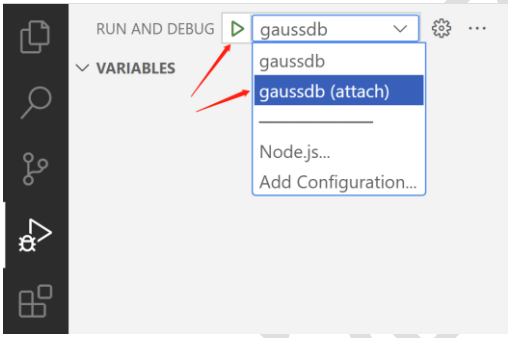

图 6.20 选择 attach 调试方式

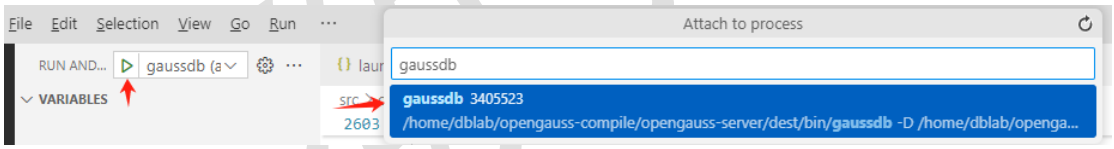

图 6.21 查找并选择 gaussdb 进程

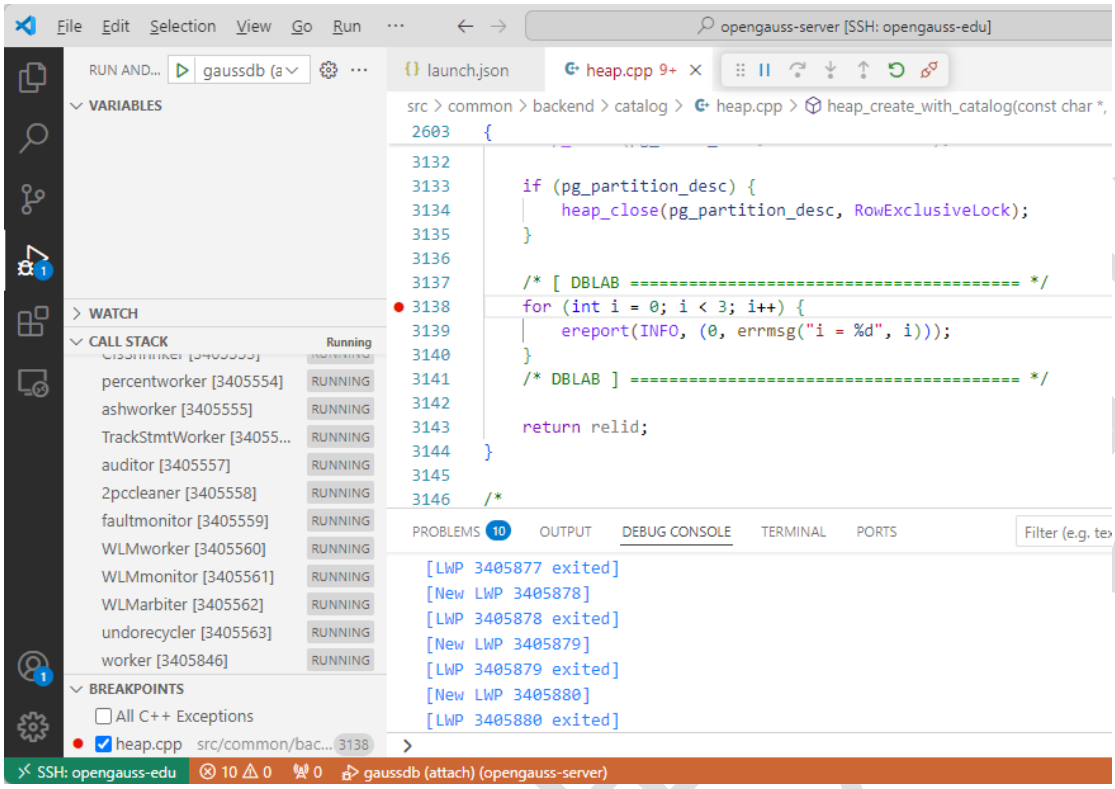

图 6.22 VS Code 以 attach 方式进入 openGauss 调试状态

5. 执行 6.4.3 节中的步骤 6、7。
6. 与 6.4.3 节步骤 8 类似，此时，VS Code 调试界面发生改变，进入了单步调试模式并停在 heap.cpp 文件中的断点位置上，如图 6.23 所示。这说明以 attach 方式调试 gaussdb 进程生效。

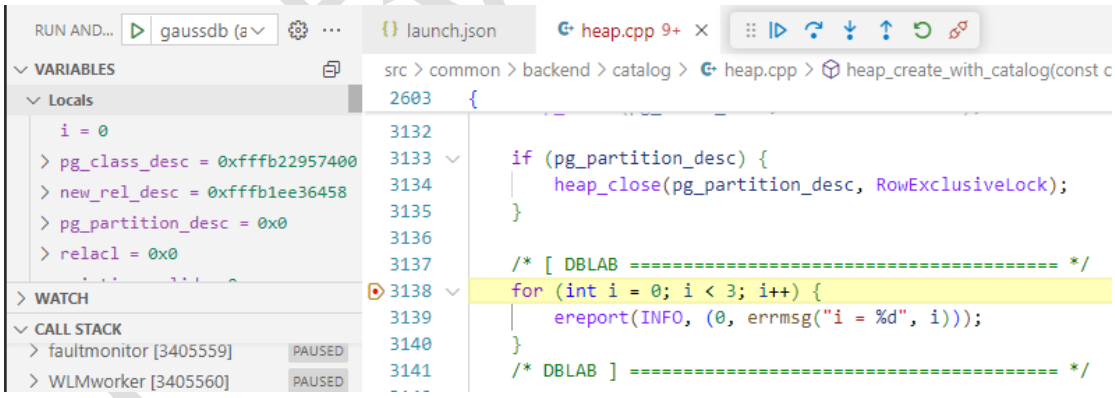

图 6.23 VS Code 以 attach 方式进行 openGauss 单步调试

7. 执行 6.4.3 节中的步骤 9、10，如图 6.24 所示。

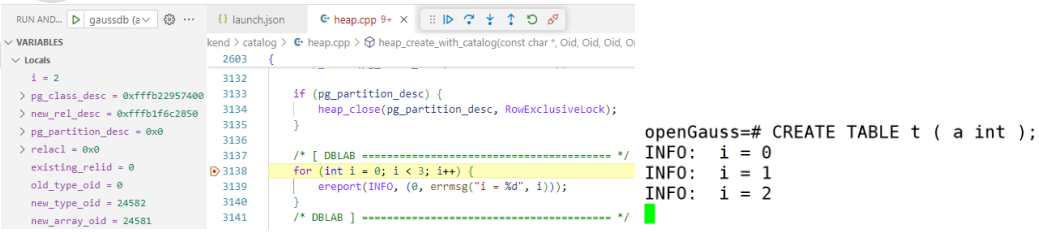

图 6.24 以 attach 方式单步调试 openGauss 代码时变量的变化以及 gsql 的输出

8. 执行 6.4.3 节中的步骤 11、12。
9. 单击调试工具栏上的“Disconnect”按钮，使 VS Code 与调试的 gaussdb 进程脱离（此操作与 attach 正好相反），结束以 attach 方式进行的调试过程。
10. 执行 6.4.3 节中的步骤 14、15、16。

Gauss数据库实验

6.5 实验结果

【请按照要求完成实验操作】

1. 完成实验步骤 6.4.1 节，在 VS Code 中查看 openGauss 数据库源代码。

2. 完成实验步骤 6.4.2 和 6.4.3，进行 openGauss 调试环境配置与单步调试，获得图 6.18 的效果。

3. 完成实验步骤 6.4.2，以 attach 方式进行 openGauss 调试，获得图 6.24 的效果。

openGauss数据库实验

6.6 讨论与总结

【请将实验中遇到的问题描述、解决办法与思考讨论列在下面，并对本实验进行总结。】

Gauss数据库实验